Sensores de PyBullet

Análisis de los Sensores

Clase de Robótica

April 9, 2026

Sensores y la "API" de PyBullet

Antes de conectar sensores, debemos entender cómo nos comunicamos con el simulador. Usamos algo llamado **API** (Interfaz de Programación de Aplicaciones).

¿Qué es una API? Imagina que vas a un restaurante. Tú (Python) quieres pedir comida, pero no puedes entrar a la cocina (el motor físico en C++ de PyBullet). La API es el **garzón**: tú le entregas comandos simples usando Python, él va a la cocina, hace el trabajo pesado y te trae el resultado.

¡Concepto Clave! A diferencia de otros simuladores, PyBullet no configura los sensores directamente en el diseño del robot. Usamos el "garzón" (la API en Python) para crear y leer los sensores en tiempo real.

¿Cómo agregamos un sensor en la práctica?

Como PyBullet maneja los sensores por código, el proceso tiene dos partes:

1. **En el diseño físico (URDF):** Creamos un *Link* (eslabón) imaginario. Le damos masa cero para que no afecte el peso, y lo atornillamos (Joint fijo) donde queremos que vaya el sensor (por ejemplo, en la "frente" del robot para una cámara).
2. **En el código (Python):** Le pedimos a la API que lea exactamente dónde está ese eslabón en el espacio 3D y, desde esa coordenada, disparamos rayos láser (LiDAR) o capturamos imágenes.

¿Dónde estoy? vs ¿Qué estoy haciendo?

Antes de leer los motores, recordemos cómo leíamos la posición del robot entero y veamos la diferencia:

1. **El "GPS" del Robot:** `getBasePositionAndOrientation`
   - **¿Qué mide?** La posición de la base o chasis en el mundo.
   - **Referencia:** El origen de la habitación virtual.
   - **Ejemplo:** "Estoy en la coordenada X: 2, Y: 5 del mapa".
2. **El "Tablero" del Robot:** `getJointState`
   - **¿Qué mide?** El estado de **un solo motor o articulación**.
   - **Referencia:** La propia mecánica del robot (propiocepción).
   - **Ejemplo:** "Mi rueda derecha ha girado 45 grados". ¡El motor no tiene idea de en qué parte del mapa está!

Llamando al "Garzón": Configurando PyBullet

Antes de pedirle datos a los sensores, tenemos que "abrir el restaurante" y llamar a nuestro garzón. En Python, esto significa importar la librería y conectarnos al motor físico.

```
import pybullet as p
import pybullet_data

# 1. Nos conectamos al simulador (Abrimos el restaurante)
cliente = p.connect(p.GUI)
```

¿Qué está pasando aquí?

- `import pybullet as p`: Traemos todas las herramientas del simulador y le ponemos el apodo `p`. A partir de ahora, cada vez que escribamos `p.`, le estamos dando una orden a la API.
- `p.connect(p.GUI)`: Inicia la simulación con una ventana gráfica para que podamos ver el entorno en 3D. (Si usamos `p.DIRECT`, simula en el fondo sin gastar recursos gráficos).

Cargando el Mundo y el Robot

Una vez que el simulador está corriendo, necesitamos poner a nuestro robot adentro para poder leer sus sensores.

```
# 2. Le decimos a PyBullet donde buscar archivos extra
p.setAdditionalSearchPath(pybullet_data.getDataPath())

# 3. Cargamos un suelo plano
p.loadURDF("plane.urdf")

# 4. Cargamos nuestro robot y GUARDAMOS SU ID
robot_id = p.loadURDF("mi_robot.urdf", [0, 0, 0.1])
```

El secreto más importante: `robot_id`
Al cargar el archivo URDF de nuestro robot, PyBullet lo mete al mundo virtual y le asigna un "número de carnet" (ID). **¡Debemos guardar este número!** Lo usaremos siempre que queramos leer un sensor de *ese* robot en específico.

Antes de leer: ¿A quién le vamos a preguntar?

Para leer un sensor interno, la API necesita saber **exactamente qué parte del robot** queremos medir. Para eso usamos el parámetro `jointIndex`. Como vimos con nuestro robot móvil, PyBullet asigna estos números **en el mismo orden en que escribimos las etiquetas** `<joint>` **en el archivo URDF**.

Revisando la estructura de nuestro `robot_movil.urdf`:

0. `joint_rueda_dd` (Delantera Derecha)
1. `joint_rueda_di` (Delantera Izquierda)
2. `joint_rueda_td` (Trasera Derecha)
3. `joint_rueda_ti` (Trasera Izquierda)

Si queremos saber qué está haciendo la rueda delantera derecha, ¡ya sabemos que nuestro índice a consultar es el **0**!

1. Encoders: Leyendo nuestros motores

Ahora que tenemos el ID del robot y el índice de la rueda, usamos nuestro "garzón" (la API) para preguntarle su estado exacto en ese instante de la simulación.

- p = la API de PyBullet
- robot_id = el "carnet" de nuestro robot en el mundo
- 0 = el indice de la rueda delantera derecha

```
1 estado_rueda_dd = p.getJointState(robot_id, 0)
```

¿Qué hace esta línea exactamente? Va al motor físico, busca al robot correcto, revisa el motor número 0 y nos devuelve una "caja" (una tupla en Python) llena de información física sobre ese motor. Toda esa caja se guarda en la variable `estado_rueda_dd`.

Desarmando la respuesta del Encoder

La variable `estado_rueda_dd` contiene varios datos mezclados. Para poder usarlos en nuestra lógica de control, tenemos que extraerlos en orden:

```
# Extrayendo los datos de la tupla (posiciones 0 y 1)
posicion = estado_rueda_dd[0]
velocidad = estado_rueda_dd[1]

print("La rueda ha girado:", posicion, "radianes")
```

¿Qué significan estos datos para una rueda?

- `estado[0]` (**Posición**): Nos da el **ángulo acumulado** en radianes. Si la rueda dio una vuelta completa hacia adelante, este valor será aproximadamente 6.28 (2π).
- `estado[1]` (**Velocidad**): Nos dice qué tan rápido está girando en ese milisegundo (radianes por segundo). ¡Ideal para saber si el robot está acelerando o frenando!

¿Por qué usamos [0] y [1]?

Cuando usamos `p.getJointState`, el "garzón" (la API) no nos trae un solo dato, nos entrega un paquete completo. En Python, este paquete se llama **Tupla** (una lista de datos que no se puede modificar).
PyBullet siempre nos entrega una tupla con **4 elementos** en un orden estricto. Imaginemos que `estado_rueda_dd` es una caja con 4 compartimentos numerados (¡en programación siempre empezamos a contar desde cero!):

- **Compartimento [0]:** Posición de la articulación.
- **Compartimento [1]:** Velocidad de la articulación.
- **Compartimento [2]:** Fuerzas de reacción en el eje.
- **Compartimento [3]:** Torque (fuerza) aplicado por el motor.

Extrayendo lo que nos importa

Si intentamos imprimir toda la caja (`estado_rueda_dd`) junta, veremos un montón de números mezclados que no nos sirven para calcular nada.
Para extraer un dato específico, escribimos el nombre de la variable seguido de corchetes `[]` con el número del compartimento que queremos leer:

```
# estado_rueda_dd = (posicion, velocidad, fuerzas, torque)

# Queremos el primer dato de la tupla (indice 0)
posicion = estado_rueda_dd[0]

# Queremos el segundo dato de la tupla (indice 1)
velocidad = estado_rueda_dd[1]
```

Nota: Por ahora solo usaremos `[0]` y `[1]` para saber dónde está la rueda y qué tan rápido gira. Los índices `[2]` y `[3]` se usan en robótica avanzada para saber si el robot está haciendo demasiada fuerza y está a punto de romper algo.

2. Sensores de Fuerza y Torque

Si queremos que el robot sepa si está chocando contra una pared o si está levantando algo pesado, necesitamos medir la fuerza. Por defecto vienen desactivados para ahorrar recursos.

```
# 1. Habilitar el sensor en el joint especifico
p.enableJointForceTorqueSensor(robot_id, joint_index, True)

# 2. Leer los datos (en el bucle de simulacion)
estado = p.getJointState(robot_id, joint_index)
fuerzas_reaccion = estado[2] # Tupla (Fx, Fy, Fz, Mx, My, Mz
    )
```

Los datos de fuerza `estado[2]` nos entregan 6 valores correspondientes a las fuerzas en los ejes X, Y, Z y los momentos (torques) en esos mismos ejes.

¿Qué significan esos 6 valores?

La tupla `fuerzas_reaccion` actúa como una celda de carga 3D instalada en la articulación de nuestra rueda. Nos entrega 6 números:

Fuerzas Lineales (Newtons):

- `Fx` (Índice 0): Fuerza en el eje de avance (Choques frontales).
- `Fy` (Índice 1): Fuerza lateral (Si chocamos de lado o derrapamos).
- `Fz` (Índice 2): Fuerza vertical (El peso del robot contra el piso).

Torques o Momentos (Newton-Metro):

- `Mx`, `My`, `Mz` (Índices 3, 4 y 5): Miden la "torsión" en esos ejes. Si la rueda se atasca, el torque subirá enormemente.

El Choque y la Tercera Ley de Newton

Imaginemos que enviamos a nuestro robot directo contra un muro a 10 rad/s. Al chocar, la consola nos mostrará valores extremos en el eje X (`Fx`). **¿Por qué?**

Aquí entra la **Tercera Ley de Newton (Acción y Reacción)**:

- **Acción:** El motor intenta seguir girando la rueda para avanzar, empujando con toda su fuerza contra la pared.
- **Reacción:** La pared virtual (que es un objeto de masa infinita y no se mueve) devuelve exactamente la misma magnitud de fuerza en sentido contrario.

El sensor **no mide el muro**, mide esa fuerza de reacción que el muro ejerce sobre la rueda intentando empujarla hacia atrás.

El efecto "Resorte" de la Simulación

Si miramos los datos del choque en la consola, veremos que saltan violentamente: `75 N`, `-79 N`, `153 N`... **¿Por qué vibran si el muro está quieto?**

En la vida real, los plásticos y las gomas se aplastan al chocar, disipando energía. **En PyBullet, los objetos son cuerpos rígidos perfectos.**

1. El robot avanza y "atraviesa" la pared un milímetro.
2. El simulador lo detecta y aplica una fuerza negativa gigante para expulsarlo de la pared.
3. El robot sale hacia atrás, pero el motor lo vuelve a lanzar hacia adelante.
4. ¡Vuelve a chocar!

Esto ocurre 240 veces por segundo. El robot está rebotando microscópicamente. En robótica, a este ruido se le llama **Chattering**.

Sensores de Fuerza y Torque

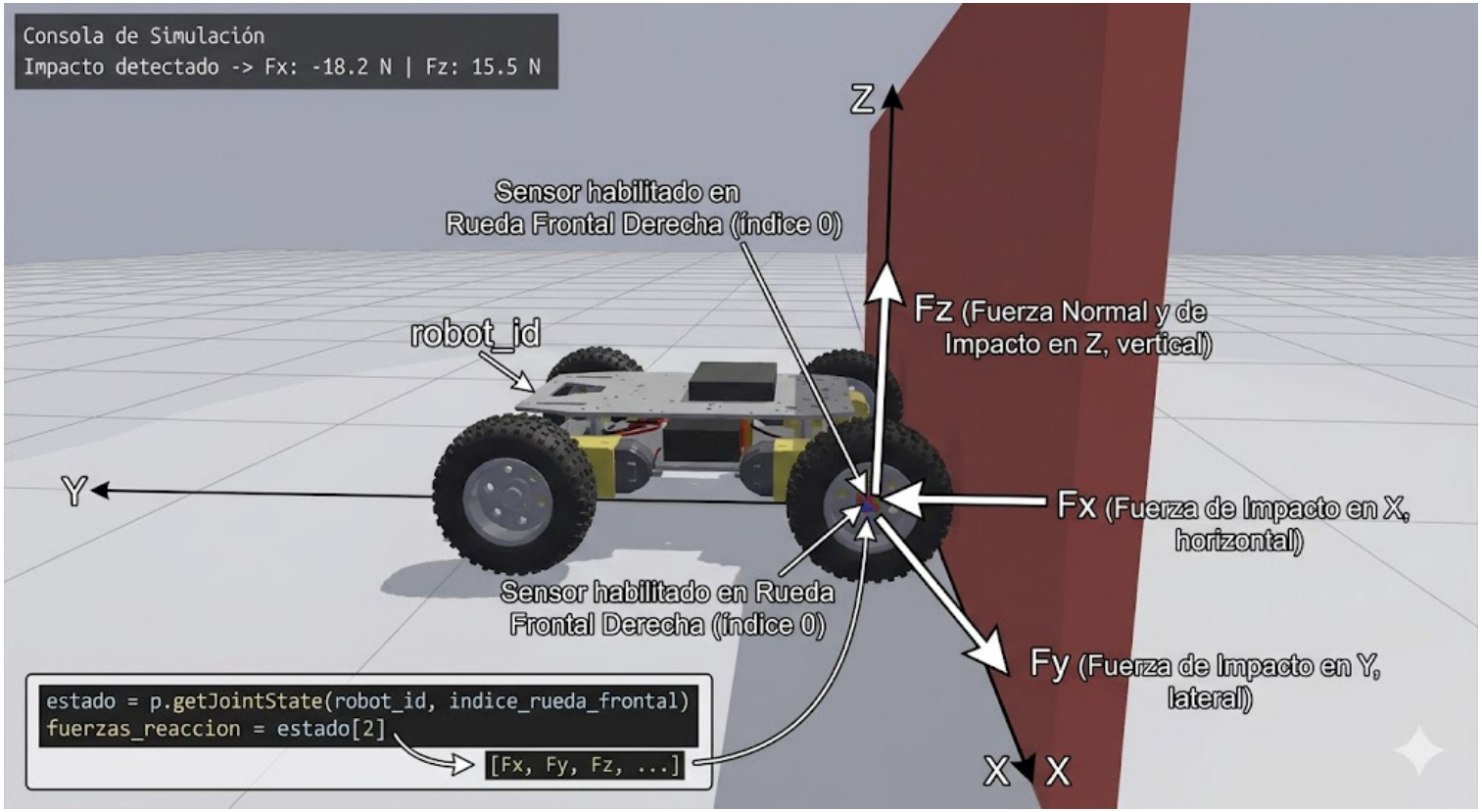

Figure: Visualización de los vectores de Fuerza

3. Sensores de Distancia (LiDAR y Ultrasonido)

PyBullet no tiene una "cámara LiDAR" o un "sensor ultrasónico" prefabricado. Para que el robot "vea" a qué distancia están los objetos, usamos una técnica matemática llamada **Raycasting** (Lanzamiento de Rayos).

¿Qué es el Raycasting?
Imagina que el robot tiene un puntero láser. El Raycasting consiste en disparar un láser virtual desde el robot hacia una dirección específica. PyBullet hace la matemática y nos avisa si esa línea imaginaria choca con la "caja de colisión" de algún objeto en el mundo.

Nota: Un sensor ultrasónico dispara un rayo (o un cono). Un LiDAR moderno dispara miles de estos rayos al mismo tiempo mientras gira, creando un mapa 3D.

Configurando el Láser Virtual

Para disparar este láser, PyBullet necesita dos coordenadas exactas en el espacio 3D: **De dónde sale** y **hasta dónde llega**.

```
# Coordenadas (X, Y, Z) en metros
origen = [0, 0, 0.5]  # El "ojo" de nuestro robot
destino = [10, 0, 0.5] # El punto final imaginario del laser

# Disparamos el rayo
resultado = p.rayTest(origen, destino)
```

Importante: El `destino` NO es el objeto que queremos chocar (¡el robot aún no sabe que hay ahí!). Es el límite de la "visión" del sensor. Si tu sensor real lee hasta 10 metros, tu punto destino debe estar matemáticamente a 10 metros del origen.

Desarmando la respuesta del Rayo

La función `p.rayTest` nos devuelve una lista. ¿Por qué? Porque PyBullet está preparado para lanzar cientos de rayos a la vez (simulando un LiDAR real). Como solo lanzamos uno, sacamos el primer elemento de la lista:

```
# Extraemos el primer (y unico) rayo de la lista
info_impacto = resultado[0]

# info_impacto es una Tupla con 5 datos. Extraeremos 3:
objeto_id = info_impacto[0]      # Que chocamos
fraccion = info_impacto[2]       # Donde chocamos (Porcentaje)
pos_impacto = info_impacto[3]    # Donde chocamos (Coordenadas)
```

Interpretando los Datos: ¿Qué chocamos?

`objeto_id` **(Índice 0):** Nos entrega el número de "carnet" (ID) del objeto contra el que chocó el láser.

- Si choca con la pared que creamos antes, devolverá el ID de la pared.
- **¿Qué pasa si no choca con nada?** Si el láser llega hasta el final de los 10 metros sin tocar nada, PyBullet nos devuelve `-1`. Este es el código universal para "camino despejado".

Interpretando los Datos: La Fracción (Hit Fraction)

PyBullet **no nos dice directamente la distancia en metros**. Por eficiencia matemática, los motores físicos calculan el impacto como un porcentaje del trayecto total. A esto le llamamos la **Fracción** (`info_impacto[2]`).

Imagina el láser como una barra de progreso del `0.0` al `1.0`:

- `0.0`: El objeto está pegado al sensor mismo.
- `0.5`: El rayo chocó exactamente a la mitad del camino.
- `1.0`: El rayo no chocó con nada (o tocó algo justo en el milímetro final).

Convirtiendo la Fracción a Metros Reales

Para que nuestro algoritmo de autonomía funcione, necesitamos transformar ese porcentaje en una distancia real en metros. La fórmula es muy sencilla:

$$\text{Distancia Real} = \text{Largo Máximo del Rayo} \times \text{Fracción}$$

Ejemplo en Python:

```
largo_rayo = 10.0 # Nuestro laser mide 10 metros
fraccion = info_impacto[2]

distancia_metros = largo_rayo * fraccion

print("El obstaculo esta a:", distancia_metros, "metros")
# Si fraccion es 0.3, la distancia sera 3.0 metros.
```

Interpretando los Datos: Coordenada Exacta

Finalmente, `pos_impacto` **(Índice 3)** nos devuelve una tupla (X, Y, Z) con la coordenada exacta del mundo donde el láser dibujó su "puntito rojo".

¿Para qué nos sirve esto?
Es la base del **Mapeo (SLAM)**. Si sabemos dónde estamos nosotros (GPS/Odometría), y sabemos la coordenada exacta del punto rojo del láser, podemos empezar a dibujar un mapa virtual de obstáculos en la memoria del robot, recordando dónde hay paredes para no chocar en el futuro.

De 1 Rayo a 360 Grados (LiDAR)

Un LiDAR real es básicamente un sensor láser (como el que acabamos de programar) montado sobre un motor que gira a altísima velocidad.

En simulación, no hacemos girar un sensor físicamente. Lo que hacemos es **lanzar múltiples rayos al mismo tiempo** en todas las direcciones, formando un círculo o un disco alrededor del robot.

El problema: Si usamos un bucle `for` para disparar 360 rayos uno por uno en cada paso de simulación, el juego se pondrá lentísimo. Necesitamos un superpoder de PyBullet.

El Súper Comando: rayTestBatch

Para resolver el problema del rendimiento, PyBullet tiene la función `rayTestBatch`. En lugar de pasarle un origen y un destino, le pasamos una **Lista de Orígenes** y una **Lista de Destinos**.

PyBullet toma esas listas, las envía al motor de C++ (que es ultrarrápido), calcula los 360 choques casi instantáneamente, y nos devuelve una **Lista de Resultados**.

```
# En vez de esto:
# resultado = p.rayTest(origen, destino)

# Hacemos esto:
resultados = p.rayTestBatch(lista_origenes, lista_destinos)
```

La Matemática del Círculo (Trigonometría)

¿Cómo calculamos los destinos para que formen un círculo perfecto?
Usamos Seno (`sin`) y Coseno (`cos`).
Si queremos lanzar N rayos, dividimos 360° (2π radianes) entre N. Luego, para cada rayo calculamos su punto final:

```
import math

# Formula para el punto final de cada rayo:
X_destino = X_robot + (Rango * math.cos(angulo))
Y_destino = Y_robot + (Rango * math.sin(angulo))
```

La Matemática del Círculo (Trigonometría)

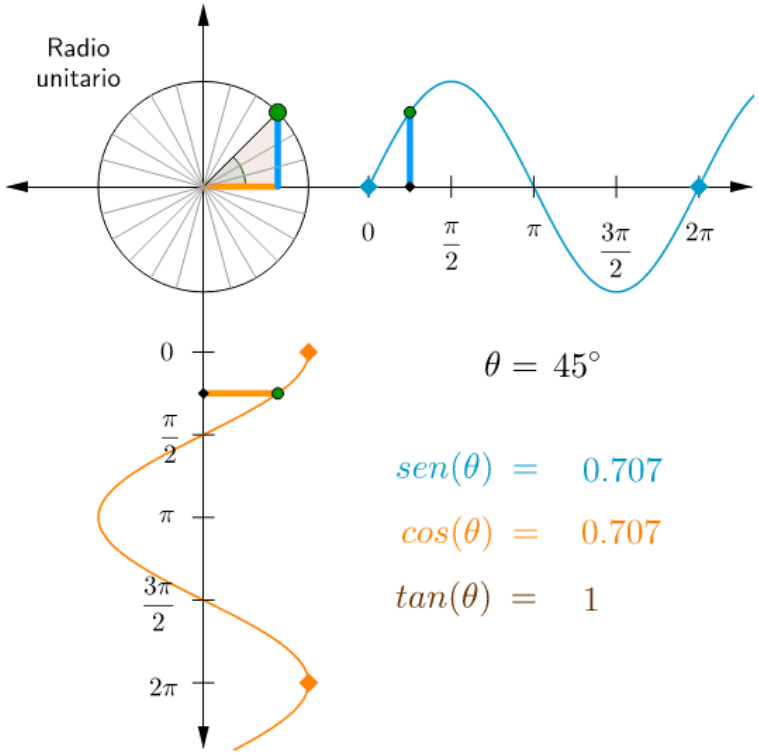

Figure: La Matemática del Círculo (Trigonometría)

Demostración

¿Qué obtenemos del LiDAR 360?

Al usar `rayTestBatch`, recibimos una lista inmensa de datos. Si lanzamos 64 rayos, recibimos 64 tuplas de información de impacto.

Nube de Puntos (Point Cloud)
Al juntar todas las coordenadas de impacto (`pos_impacto`) de los rayos que sí chocaron con algo, obtenemos una "silueta" del entorno. El robot sigue ciego, pero ahora tiene un "mapa de colisiones" en su memoria que le dice exactamente dónde están las paredes a su alrededor. ¡Esta es la base del SLAM!